



What is SQL & how web apps use it

- SQL = Structured Query Language. The language apps use to talk to databases.
- Web flow: Browser → Server → Database → Server → Browser.
- `SELECT * FROM users WHERE username = 'ali';`
- Analogy: Database is a library; SQL is how you ask the librarian for specific books.
- Web apps use SQL to store user data, read data, and update/delete data.



What is SQL Injection (SQLi)?

- SQLi is when an attacker inserts (injects) malicious input into an SQL query so the database executes unwanted commands.
- user input changes the meaning of a query.
- “Attacker tricks the app into running a different SQL command.”
- In a login form, attacker types something that makes the database say “yes” for any password.

Why does SQLi happen? **(Root causes)**

- App directly inserts user input into SQL strings.
- Concatenating strings to build queries (e.g., "... WHERE name = '" + user + "'").
- App uses a DB account with too many rights (can DROP tables).
- Detailed DB errors reveal structure and help attackers.
- Poor validation & encoding: No checks on type, length, characters of input.

Types of SQL Injection

- **In-band SQLi:** attacker uses same channel to send and receive data.
- **Error-based:** Force DB errors to reveal info.
- **Example:** entering malformed input to see database error message.
- **Analogy:** Ask librarian a question that makes them shout the shelf names.
- **UNION-based:** Use UNION to append attacker query and return extra rows.
- **Example payload idea:** ...' UNION SELECT username, password FROM users --
- **Analogy:** Adding your own page to the librarian's response that lists secret books.

- **Blind SQLi:** app doesn't return DB data directly; attacker infers info.
- **Boolean-based:** Send queries that return true/false, watch page differences.
- **Analogy:** Ask yes/no questions and infer answer by reaction.
- **Time-based:** Use delays (sleep) to infer true/false by how long response takes.
- **Analogy:** If librarian pauses for 5 seconds, answer was "yes."

- **Out-of-band (OOB) SQLi:** attacker forces DB to call an external server to leak data.
- **Example:** making DB connect to attacker.com with secret in URL.
- **Analogy:** Librarian secretly sends the list to your phone number.

 **Payload**

- **Bypass login (classic):**

- `' OR '1'='1' --`

- **What it does:** Turns condition into always true so login bypasses check.

- **UNION data extract (concept):**

- `' UNION SELECT username, password FROM users --`

- **What it does:** Appends attacker query to return user table rows.

- **Blind boolean (concept):**
- `' AND (SUBSTRING(password,1,1)='a') --`
- **What it does:** Changes response only if first password character = 'a' (infers letters).
- **Time-based (concept):**
- `' OR (CASE WHEN (SUBSTRING(password,1,1)='a') THEN pg_sleep(5) ELSE pg_sleep(0) END) --`
- **What it does:** Delays response if condition true — attacker measures time to infer data.

Impact **(what an attacker can do)**

- Read sensitive data
- Modify / delete data
- Bypass authentication
- Escalate to full compromise

How to detect SQLi

- Code review: look for string concatenation with user input in SQL.
- Input fuzzing: send special characters (' , -- , ;) and observe errors or behavior changes.
- Log analysis: unusual queries, spikes in errors, abnormal response times (for time-based).
- Behavioral testing: check for differing page content on crafted inputs (boolean blind).
- Security tools: automated scanners and sqlmap (used in controlled testing) can identify weaknesses.

Prevention & Mitigation (best practices)

- Use Prepared Statements / Parameterized Queries

Example: `SELECT * FROM users WHERE username = ?` with bound parameter.

- Input Validation & Whitelisting
- Least Privilege
- Hide detailed errors
- Use ORM safely
- Escape inputs only with trusted libraries
- Web Application Firewall (WAF)
- Regular patching & dependency management
- Monitoring & Alerts